
Asynchronous Compiler-Supervised Decoding for Code Generation

Leo Wang*

Department of Computer Science
University of Chicago
psixyzt@gmail.com

Abstract

Verifier-guided code-generation systems — Monitor-Guided Decoding [1], ROCODE [12], SemGuard [22], and Type-Constrained Code Generation [16] — all share one design choice: the verifier runs *synchronously* on the decoding critical path. The decoder stalls while the language server, compiler, or learned classifier returns its verdict. This is tolerable when the verifier is fast (Python’s `compile()`, or a 1.3B classifier on one line of code), but it forecloses the use of slower, sounder oracles — `cargo check` on Rust (~35–500 ms incremental), `javac` on Java (~180 ms), per-invocation compiler subprocesses in the decoding loop (~0.55 s amortized for `g++ -fsyntax-only` in our setup), or whole-crate type and lifetime analyses taking seconds. We present *SoundCode*, a producer-consumer decoding loop in which a compiler verifier (`cargo check` for Rust, `g++ -fsyntax-only` for C++) runs *asynchronously* alongside the LLM, spawning a check at each depth-zero structural boundary (`;` or `}`) and triggering rollback to the most recent boundary checkpoint only when a returned diagnostic is blocking. The same rollback algorithm underlies the synchronous ROCODE baseline [12]; we isolate the sync-versus-async axis at fixed verifier, fixed model, and fixed benchmark. We evaluate Qwen2.5-Coder-7B-Instruct [10] on MultiPL-E HumanEval-rs and HumanEval-cpp [4]. Holding the model, the verifier, and the rollback algorithm fixed, SoundCode is $1.47\times$ faster on Rust (122/156 paired wins; async verification costs 1.4% over verifier-free decoding) and $2.16\times$ faster on C++ (159/161 paired wins) than the synchronous ROCODE-style equivalent, with consistent ratios on MBPP ($1.54\times/2.18\times$, $n=354/397$) and larger ones for a faster-decoding 1.5B model ($1.69\times/3.17\times$), as a verifier-cost-spectrum model predicts. `pass@1` is statistically indistinguishable between the two schedulings on both benchmarks (McNemar exact $p \geq 0.092$ everywhere), so our contribution is a pure *latency* result: asynchronous scheduling removes the synchronous verifier’s critical-path cost without measurably changing accuracy. Beyond the measurement, we offer a design framework — when to async, when to roll back, and where to roll back — that applies to any verifier whose cost is not negligible relative to a single token’s decoding latency. A live web demo and reference implementation are open-source.

1 Introduction

Large language models predict next tokens; they do not enforce programming-language rules. Even after billions of training tokens of high-quality code, instruction-tuned code LMs hallucinate ill-typed expressions, undefined identifiers, ill-formed borrows, and nonsensical control flow. The hallucinations are not bugs in particular models — they are a structural consequence of how autoregressive

*Code and live demo: github.com/leobwang/soundcode; soundcode-demo.psixyzt.com.

sampling commits to one token at a time without look-back: any path the model places non-trivial probability mass on becomes reachable, and a single bad token can cascade [12, 22]. Recent work on the *intrinsic* sources of hallucination in language modelling confirms this: a frozen LM that has been trained on data with even a small fraction of subtly incorrect programs will reliably generate some subtle errors at inference, because next-token cross-entropy gives no mechanism to refuse paths that violate global constraints [17].

Programming-language verifiers are exactly the missing mechanism. A compiler, a type checker, a borrow checker, or a Language Server Protocol (LSP) server, taken on the decidable fragment of code it covers — syntax, type, name resolution, lifetimes — returns a ground-truth verdict: either the program is well-formed or it is not. This verdict is decoupled from how the program was produced. It is also exactly the kind of side information a generative LM has no internal way to consume. The natural design move is to put one in the decoding loop.

Several recent systems do exactly this. Monitor-Guided Decoding (MGD; 1) consults an LSP at hand-picked syntactic triggers — the `.` that begins a member-access expression in Java — and masks the LM’s next-token logits to those prefixing a type-consistent identifier. ROCODE [12] runs the compiler incrementally after each generated statement, rolls back the buffer on an error, and decodes the retry under an exponentially-decaying token-penalty mask. SemGuard [22] swaps the compiler for a learned 1.3B semantic classifier that scores each emitted line and triggers line-level rollback below a threshold. Type-Constrained Code Generation [16] formalizes per-token soundness in TypeScript by composing a prefix automaton with a type-reachability search.

All four share one design choice: the verifier runs *synchronously* on the decoding critical path. The LM stops; the verifier returns; the LM resumes. This is invisible when the verifier is fast — Python’s `compile()` parses a partial program in ~ 0.1 ms, `g++ -fsyntax-only` parses a warm translation unit in ~ 3 ms (though its per-invocation subprocess cost inside a decoding loop is far higher — ~ 0.55 s amortized in our setup, §4.4), and a 1.3B classifier evaluates one line of context in single-digit milliseconds on a modern GPU. ROCODE’s benchmarks are Python and C++ [12], SemGuard’s are Python and Java [22], MGD’s are Java with hand-picked dereference triggers [1]. In each case the per-call verifier cost is well below the LM’s own per-token latency, so the slowdown is tolerable. MGD reports a $\sim 83\%$ mean-decoding-time slowdown [1]; Type-Constrained reports a 39–52% median per-instance overhead [16]. The cost is real but amortized away by the gain in code quality.

The same design fails to scale to slower oracles. `cargo check` on a typical Rust function takes ~ 35 –500 milliseconds incrementally (amortizing to ~ 0.20 s per call in our setup); `javac` on a single Java method takes ~ 180 ms once JVM startup is amortized; a whole-crate borrow check on an unfamiliar trait expansion can take seconds. A synchronous decoding loop that blocks on such an oracle spends most of its wall time idle. The two natural responses — (1) substitute a faster, weaker oracle, e.g. a learned classifier [22]; or (2) accept a $> 10\times$ slowdown and amortize against the larger code-quality gain — both surrender something the design originally promised. The first sacrifices soundness (a learned classifier is wrong on a defined fraction of inputs by construction); the second sacrifices the wall-clock budget the verifier-guided pipeline was meant to outperform.

This paper takes a different response: *run the verifier asynchronously*. We introduce **SoundCode**, a producer-consumer decoding loop in which:

- the LM streams tokens uninterrupted (the producer);
- at each depth-zero structural boundary — `;` or `}` at brace-depth zero in C-family languages, newline at indent zero in Python — a background task is spawned to run the verifier on the current buffer snapshot;
- a consumer task drains finished checks in FIFO order;
- on the first *blocking* diagnostic, pending checks are cancelled, the buffer is truncated to the most recent boundary checkpoint, and the LM is re-prompted from there.

Crucially, the same rollback algorithm underlies the synchronous ROCODE-style baseline that we compare against; *the only architectural difference is whether the verifier blocks the decoder*. This isolates the sync-vs-async axis at fixed verifier, fixed model, and fixed benchmark.

We evaluate SoundCode against ROCODE-style synchronous verification on HumanEval-rs ($n = 156$ after filtering) and HumanEval-cpp ($n = 161$) from MultiPL-E [4], using Qwen2.5-Coder-7B-Instruct

[10] via vLLM. We report pass@1, mean wall-clock time per problem, mean rollback count, and compile rate, plus an ablation on rollback target (naive last-checkpoint vs. a recurrence-gated back-off variant derived from ROCODE’s recurrence rule).

Headline result. Holding the model, the verifier, and the rollback algorithm fixed, SoundCode is $1.47\times$ faster on Rust (1.05 s sync $\rightarrow$ 0.71 s async; $1.31\times$ at the per-problem median; async beats sync on 122 of 156 paired problems, 78.2%) and $2.16\times$ faster on C++ (2.50 s $\rightarrow$ 1.16 s; $2.20\times$ at the median; async wins on 159 of 161 paired problems, 98.8%) than the synchronous ROCODE-style equivalent — on Rust, async verification costs just 1.4% over verifier-free decoding. pass@1, by contrast, does not separate the two schedulings: the per-language differences (+1.8 pp for async on C++, −4.5 pp on Rust) amount to a handful of problems on a single greedy run, flip sign between languages, and are not statistically significant (McNemar exact $p=0.58$ and $p=0.092$ respectively). The claim of this paper is therefore a *latency* claim, not a quality claim: asynchronous scheduling removes the synchronous verifier’s wall-clock penalty at statistically unchanged accuracy.

Contributions. We frame the contribution along four axes:

1. **The first asynchronous design for verifier-guided code generation.** Every prior boundary-rollback system [12, 22, 23] runs its verifier synchronously; every prior asynchronous-parallel-verify system [14, 5, 8] uses the LM itself as the verifier and gives up soundness. SoundCode occupies the open “semantic-verifier $\times$ structural-rollback $\times$ async” design point.
2. **A controlled head-to-head measurement.** At fixed model, fixed verifier, and fixed rollback algorithm — so the only axis that varies is whether the verifier blocks the decoder — async delivers a $1.47\times$ wall-time speedup on Rust (122/156 paired wins) and a $2.16\times$ speedup on C++ (159/161 paired wins), with pass@1 statistically indistinguishable between the schedulings on both languages, replicated across benchmarks (MBPP: $1.54\times/2.18\times$) and model sizes.
3. **A verifier-cost-spectrum framing that predicts when async helps.** We sort verifiers along a cost spectrum (Python’s `compile()` ~ 0.1 ms; `g++ -fsyntax-only` ~ 3 ms warm, amortizing to ~ 0.55 s per in-loop invocation in our setup; `cargo check` ~ 35 –500 ms, amortizing to ~ 0.20 s; `javac` ~ 180 ms; whole-crate analyses up to seconds) and argue that the async design pays off whenever per-call verifier cost exceeds per-token decoding latency. The Rust-vs-C++ asymmetry in our results — a smaller win on Rust, whose verifier amortizes cheaper in-loop (~ 0.20 s/call vs C++’s ~ 0.55 s/call) — is the first empirical confirmation of this prediction, and a follow-up model-size scaling test confirms it from the other direction: a $\sim 3.7\times$ -faster-decoding 1.5B model widens the win to $1.69\times/3.17\times$ (§4.6).
4. **A reproducible open-source implementation and live demo.** The producer is vLLM’s AsyncLLMEngine [13] run with `enforce_eager=False`; the verifier is `cargo check / g++ -fsyntax-only` via subprocess (the driver used for every headline measurement), with an LSP driver (`rust-analyzer` via `multitls` [1, 2]) evaluated in the follow-up (§4.6). Engineering effort to port to a new language is one boundary-detector and one verifier-driver class. Source: github.com/leobwang/soundcode; live demo: soundcode-demo.psixyzt.com [25].

The Olausson et al. matched-budget framework [17] sets the bar any verifier-guided system must clear: a self-repair loop only beats matched-budget i.i.d. resampling when the feedback signal is *strictly stronger* than the generator. We argue that a real compiler is exactly that. A learned classifier is not. SemGuard [22] acknowledges this trade-off explicitly. Our async design lets us pay for the strictly-stronger oracle in wall-clock terms without paying for it in throughput.

2 Related Work

SoundCode sits at the intersection of three lines of work: (i) decoding-time methods that consult a programming-language verifier and react to its output — the closest neighbours, treated in detail below; (ii) per-token constrained-decoding methods that mask the LM’s logit vector against a grammar or type lattice but do not roll back; and (iii) speculative-decoding methods that run a verifier

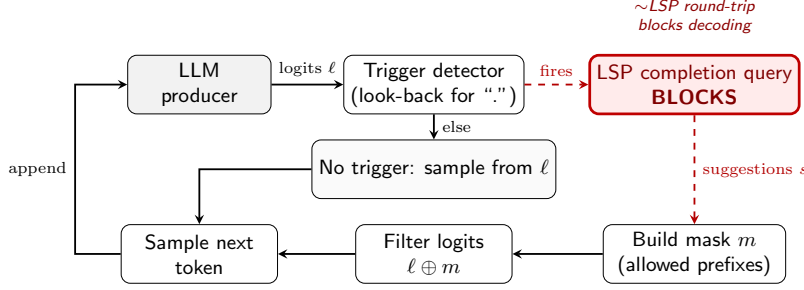


Figure 1: Monitor-Guided Decoding [1]: the LSP is queried at each trigger token (. in Java); tokens outside the returned suggestion set are masked to $-\infty$. The LSP round-trip blocks decoding at every triggered token.

asynchronously alongside the LM, but use the LM itself as the verifier. SoundCode borrows the rollback algorithm from (i), the mask-when-cheap idea from (ii), and the parallel-verify control flow from (iii); its open contribution is to combine all three.

2.1 Verifier-in-the-loop decoding (the closest neighbours)

Monitor-Guided Decoding (MGD). Agrawal et al. [1] introduce a framework in which a stateful *monitor* sits between the LM and a Language Server Protocol (LSP) instance. The monitor’s pre-condition fires on a trigger token (in the canonical instantiation, the . that begins a Java member-access expression). On firing, the LSP returns the type-consistent identifiers reachable at that source location; the monitor converts the set to a binary mask over the LM vocabulary, zeroes the logits of tokens not prefixing any allowed identifier, and resumes decoding. Each tokenization step that lands on a trigger pays a synchronous LSP round-trip. The reported mean-decoding-time slowdown is approximately 83% [1, Table 2] on Java DOTPROMPTS, traded for a +18% to +25% relative compilation-rate gain across CodeGen-350M to text-davinci-003. The monitor does not roll back: if a triggered LSP query returns an empty allowed set, the run is abandoned. MGD is the canonical *prevention by masking* design point.

ROCODE. Jiang et al. [12] compile-check after every emitted statement, roll back on a detected error, and decode the retry under an exponentially-decaying token-penalty mask. Two rollback rules coexist: r_e , the error-line offset reported by the compiler, is tried first; if the compiler error recurs at the same line, the system falls back to r_h , the line containing the highest-entropy token in the prefix [12, Eqs. 5–7]. A Trie tracks attempted paths so previously-failed token suffixes accumulate decayed penalties on each retry. ROCODE targets Python and C++ and reports a PassRate of 57.3 on HumanEval(ET) with CodeLlama-7B at $p=0.9$ nucleus, +30.6 pp over the unconstrained baseline and +11.0 pp over PG-TD. Crucially, the verifier runs synchronously between statements; the design only works because Python’s `compile()` returns in ~ 0.1 ms and `g++ -fsyntax-only` in ~ 3 ms. ROCODE is the closest direct competitor to SoundCode.

SemGuard. Tian et al. [22] train a 1.3B LLM-based binary classifier on a curated diff corpus (SemDiff) that pairs correct and incorrect submissions to the same problem from CodeNet. After every emitted line, the classifier scores the partial program; if the score drops below 0.5, the LM rolls back to the start of the line and resamples under a single-token penalty on the offending first non-indent token, retrying up to N times. SemGuard targets Python and Java and reports a +2.23 pp gain over ROCODE on SemDiff with DeepSeek-Coder-6.7B, at 31% fewer tokens and 60% lower wall-clock [22, Tables 3, 6]. The verifier is again synchronous, but the per-line classifier-pass cost (single-digit ms on a modern GPU) is comparable to ROCODE’s `compile()`-based version. The verifier is *learned*, so soundness is empirical rather than guaranteed — SemGuard reports per-task false-positive rates centred on 0.26–0.34.

IterGen. Ugare et al. [23] target grammar-constrained structured-output generation (SQL, Vega-Lite JSON, regex). They expose a Python API of `forward(symbol, n)`, `backward(symbol, n)`, and `view(symbol)` primitives indexed by grammar non-terminals, and implement backward via

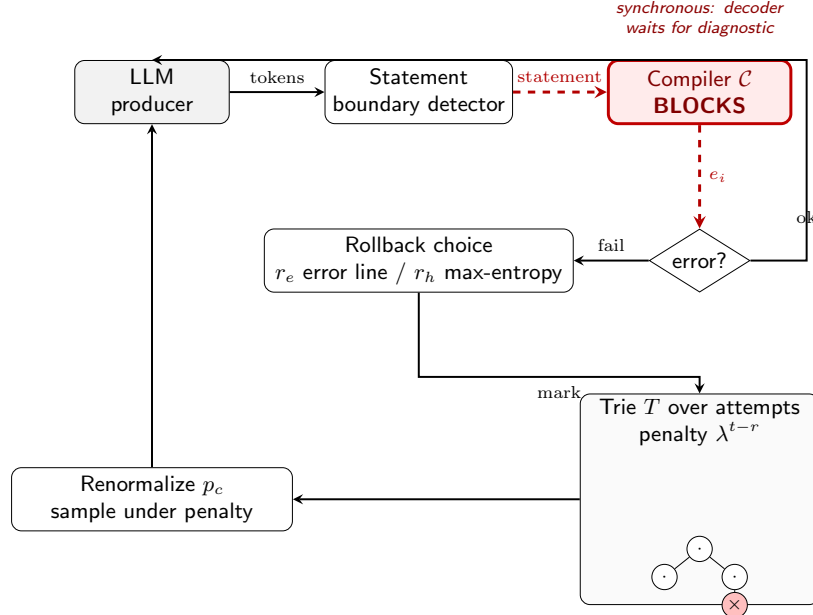


Figure 2: ROCODE [12]: the compiler runs synchronously after each emitted statement; on a detected error, the buffer is rolled back to the compiler-reported line, or, on recurrence, to the line containing the highest-entropy token. The retry decodes under an exponentially-decaying token-penalty mask.

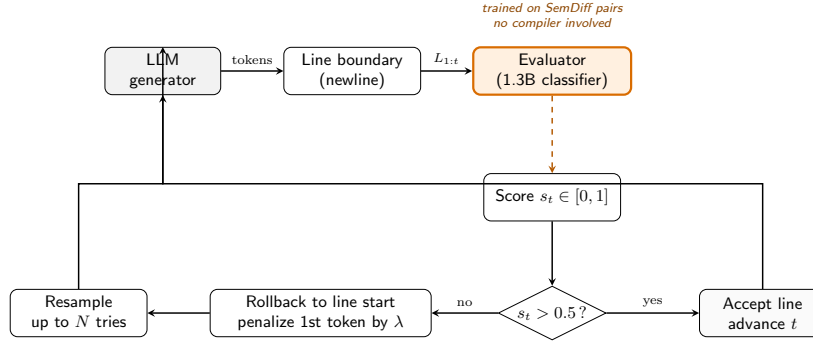


Figure 3: SemGuard [22]: a learned 1.3B semantic classifier scores each emitted line; if the score drops below 0.5, the buffer is rolled back to the start of the offending line and the LM resamples under a single-token penalty on the first non-indent token, up to N retries.

KV-cache cropping driven by a shift-reduce parser’s symbol-position map. The verifier is whatever Python predicate the user writes against `view(symbol)` — typically a schema-membership check, a regex match, or a JSON field-type assertion. Verification is synchronous between forward calls but cheap because the user predicates are typically constant-time. SoundCode borrows IterGen’s KV-cropping idea as a low-overhead rollback implementation; the verifier differs (real compiler vs. user predicate) and the target language differs (Turing-complete Rust vs. declarative DSLs).

2.2 Constrained decoding without rollback

A separate line of work prevents bad outputs by masking the LM’s logits against a grammar, type lattice, or completion engine, but never rolls back: rejected tokens never make it into the buffer in the first place. *PICARD* [20] runs a fast monadic SQL parser on each top- k beam continuation and assigns $-\infty$ to tokens whose partial detokenization fails to parse; *Synchromesh* [18] generalizes this with hand-written Completion Engines for SQL, Vega-Lite, and SMCaFlow, using Brzozowski derivatives to reconcile LM-token vs. DSL-token mismatch. *Grammar-Constrained Decoding* [9]

extends the mask to arbitrary context-free grammars; *DOMINO* [3] makes the mask near-zero-overhead via per-state prefix trees and opportunistic argmax-first verification. *Type-Constrained Code Generation* [16] extends the construction from syntax to types: a prefix automaton plus a type-reachability search enforce well-typedness of every sampled token on a simply-typed Turing-complete calculus, implemented for TypeScript. The reported median per-instance overhead is 39–52%; pass@1 rises by +3.5%—+5.0% on TypeScript HumanEval and MBPP synthesis. All of these systems are synchronous and per-token, and none rolls back; semantic errors that surface only across multiple tokens (borrow-checker violations, multi-statement type-inference failures) are out of reach for them by construction. They are most useful as *cheap front-ends* composed with SoundCode’s expensive-verifier-with-rollback back-end.

2.3 Speculative decoding: the structural blueprint

The structural insight SoundCode reuses is the *speculate-ahead, verify-in-parallel, accept-or-rollback* control flow of speculative decoding. Leviathan et al. [14] and Chen et al. [5] draft K tokens from a small fast model, then score them in a single parallel forward pass of the large target model, accepting a left-to-right prefix while provably preserving the target distribution. Reported speedups are $2.46\times$ on HumanEval with Chinchilla 70B [5] and up to $3.4\times$ on translation [14]. Fu et al. [8] achieve a draft-model-free variant by reformulating decoding as a Jacobi fixed-point and verifying cached n -grams in parallel inside the same model, reaching $1.5\times$ – $2.3\times$ on a single A100 and up to $4\times$ with Lookahead Parallelism on 8 A100s. SoundCode swaps the LM-as-verifier for a *semantic* verifier (`cargo check / g++ -fsyntax-only`) and replaces per-token rollback with structural-boundary rollback. The control-flow skeleton — producer runs ahead, verifier runs in parallel, on rejection rollback to a prior accept — is the same.

2.4 Iterative repair (the contrast)

A sibling family runs the verifier *off* the critical path entirely. Self-Refine [15], Reflexion [21], Self-Debug [7], INTERVENOR [24], and AlphaCodium [19] all generate a whole candidate solution, run the verifier on the finished artifact, and condition a fresh generation on the verifier’s natural-language output. These pay for the verifier only once per solution but discard a long completion on each fix attempt. The pivotal methodological result is Olausson et al. [17]’s observation that, across HumanEval and APPS, self-repair beats matched-budget i.i.d. resampling only when the feedback model is *strictly stronger* than the generator — a same-model self-critique loop roughly cancels its own compute cost. SoundCode is designed to clear this bar: `cargo check` is strictly stronger than the generator on the decidable fragment of syntax, type, and lifetime correctness.

2.5 Benchmarks and base models

We evaluate on MultiPL-E HumanEval-rs and HumanEval-cpp [4], two of the multilingual ports of the original HumanEval benchmark [6]; the source problems are 164 hand-written Python signatures plus docstrings, mechanically translated to Rust and C++ with native-language unit tests preserved (156 Rust and 161 C++ translations survive the benchmark’s filtering). The base model is Qwen2.5-Coder-7B-Instruct [10], a frontier open-weight code LM in the 7B class.

2.6 Synthesis

The four-way SoundCode framing follows from the survey: every decoding-time-with-verifier system in Section 2.1 runs synchronously; every async parallel-verify system in Section 2.3 uses the LM as verifier. The combination semantic-verifier $\times$ structural-rollback $\times$ asynchronous-on-the-critical-path is the open design point. The choice of verifier (real compiler / LSP vs. learned classifier vs. grammar) is forced once one commits to that combination: a learned classifier is too weak by Olausson’s criterion; a grammar is too weak for type and borrow correctness; a compiler is strong enough, but its latency makes synchronous use impractical, so the async design is what unlocks the use of the strictly-stronger oracle.

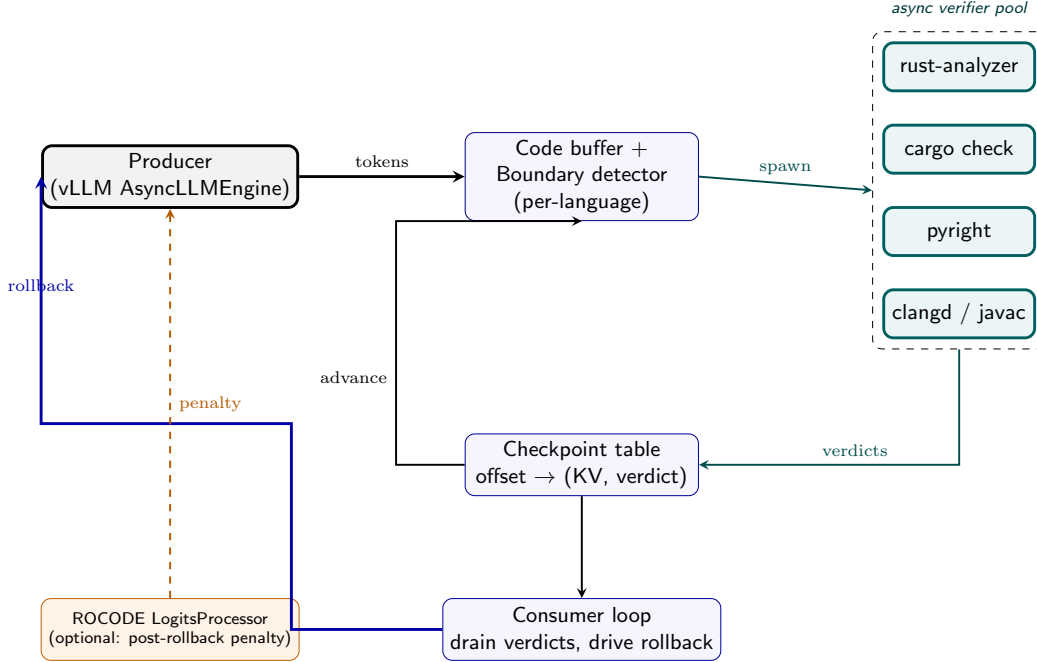


Figure 4: SoundCode’s producer-consumer architecture. The producer LM streams tokens; a structural boundary detector identifies depth-zero `;` and `}` in the function body; at each boundary an async verifier task is spawned on a snapshot of the current buffer; a consumer drains finished verifier tasks in FIFO order; on the first blocking diagnostic, pending tasks are cancelled and the buffer is rolled back to the most recent boundary checkpoint.

3 Method

We describe SoundCode along four axes. Section 3.1 gives the system architecture and identifies the four components that change relative to a synchronous baseline. Section 3.2 presents the producer-consumer decoding loop in pseudocode. Sections 3.3–3.5 answer the three design questions inherited from RCODE [12] and reframed by the async setting: *when* to run the verifier asynchronously, *when* to roll back, and *where* to roll back to. Section 3.6 summarizes the implementation.

3.1 System architecture

Four components change relative to the synchronous baseline:

1. A **structural boundary detector** that, given the running token buffer, identifies the points at which a verifier call should be triggered. We use depth-zero `;` and `}` for the C-family languages (Rust, C++, Java); newline at indent zero for Python. The detector is language-specific but stateless across generations.
2. An **async verifier task** that, given a snapshot of the buffer at a boundary, runs the compiler / LSP server and returns a diagnostic classified as `BLOCKING`, `INCOMPLETE`, or `NONBLOCKING`. Tasks are spawned at boundary time and never share state; `cargo check` reuses the workspace’s incremental cache across calls.
3. A **verifier queue** of in-flight tasks, ordered by spawn time. The consumer polls this queue on a tight schedule (every K tokens; we use $K = 1$) and removes completed tasks left to right.
4. A **rollback driver** that, on the first `BLOCKING` diagnostic, cancels pending tasks, truncates the buffer to the most recent boundary checkpoint (backing off to earlier checkpoints on successive retries), and re-prompts the LM from that prefix.

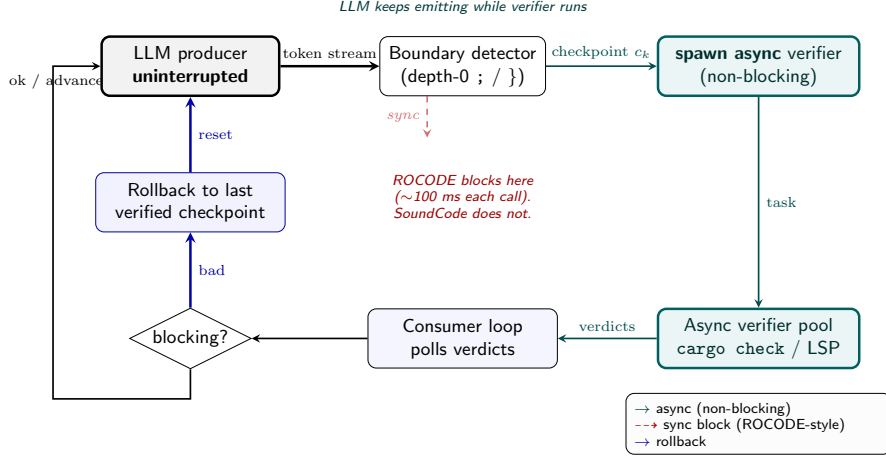


Figure 5: SoundCode workflow. The LM (top, black) streams tokens. At each depth-zero `;/}` boundary, an async verifier task (teal, dashed) is spawned on a snapshot of the buffer. The decoder *does not block* on the verifier — where ROCODE [12] would (ghost red arrow, omitted in SoundCode), control returns immediately to the producer. The consumer (blue) drains finished verifier tasks in FIFO order. On the first BLOCKING diagnostic, pending tasks are cancelled and the buffer is rolled back to the most recent boundary checkpoint.

The producer task is the LM, unmodified except for an `abort()` hook that lets the rollback driver interrupt an in-progress generation. We run inference with vLLM [13]’s `AsyncLLMEngine`, which natively supports per-request abort and KV-cache reuse for the surviving prefix; the rollback amounts to discarding the abandoned suffix and resubmitting the truncated buffer as a fresh `generate()` call.

At end of stream, the consumer awaits any in-flight verifier tasks before returning, and a trailing blocking verdict still triggers rollback and regeneration. Note that checkpoints are recorded eagerly at boundary time (§3.5), so mid-stream checkpoints are *candidate* rollback targets rather than verifier-cleared positions; final well-formedness is established by the end-of-stream drain plus the post-hoc compile-and-test evaluation.

3.2 Algorithm

Algorithm 1 states the decoding loop. We follow the ROCODE Algorithm-2 structure [12]; the critical difference is that lines 7 and 13 run as separate coroutines rather than as sequential statements, and the rollback test at line 15 acts on already-completed verifier tasks rather than blocking on the just-scheduled one.

The algorithm is parameterized by the rollback policy π , which maps a (buffer, diagnostic, checkpoint set) triple to a token offset. The concrete instantiations are defined in Section 3.5: `NAIVELAST` (return $\max C$), `RECBACKOFF` (`NAIVELAST`, but back off one additional checkpoint when the same error recurs at the same line), `ROCODEENTROPY` (try error-line offset, fall back to max-entropy on recurrence), and `NAIVELAST+PENALTY` (return $\max C$ and add ROCODE’s λ^{t-r} token penalty to the retry).

3.3 When to async: a verifier-cost spectrum

The motivating constraint is that asynchronous verification only pays off when the verifier is slow enough that the per-token generation budget is dominated by verifier idle time. Table 1 sorts the verifiers used by the closest-neighbour systems along this axis.

Rule of thumb. If a single verifier call costs less than a single LM token’s wall time, synchronous verification is the right design — the synchronization overhead of spawning, polling, and awaiting will exceed the saved blocking time, and the simpler architecture is preferred. ROCODE’s choice of synchronous integration is correct in the millisecond-verifier regime it measured (Python’s

Algorithm 1: SoundCode asynchronous compiler-supervised decoding.

Input: prompt x ; LM $\mathcal{M}$; verifier $\mathcal{V}$; boundary detector $\mathcal{B}$; rollback policy π **Output:** verified completion y

```
1 queue  $Q \leftarrow \emptyset$ ;
2 checkpoints  $C \leftarrow \{0\}$ ; // boundary token indices (eager)
3 buffer  $y \leftarrow \text{empty}$ ;
4 spawn producer: while not EOS and not aborted do
5    $t \leftarrow \mathcal{M}.\text{NEXT}(x, y)$ ;
6    $y \leftarrow y \cdot t$ ;
7   if  $\mathcal{B}(y)$  then // e.g. depth-0 ; or }
8      $C \leftarrow C \cup \{|y|\}$ ; // checkpoint recorded at boundary
9      $Q.\text{PUSH}(\text{SPAWN-ASYNC } \mathcal{V}(y), \text{idx} = |y|)$ ;
10  end
11 end
12 spawn consumer: while producer running or  $Q \neq \emptyset$  do
13   while  $Q$  has a completed head  $h$  do
14      $d \leftarrow h.\text{RESULT}$ ;
15     if  $d$  is BLOCKING then
16       cancel pending tasks;  $Q \leftarrow \emptyset$ ;
17        $\mathcal{M}.\text{ABORT}$ ;
18        $r \leftarrow \pi(y, d, C)$ ; // rollback target (§3.5)
19        $y \leftarrow y[:r]$ ;  $C \leftarrow \{c \in C : c < r\}$ ; // pop used checkpoint
20       restart producer with prompt  $x \cdot y$ ;
21     else if  $d$  is NONBLOCKING then
22       // no state change -- checkpoint was recorded at boundary
23   end
24   YIELD; // cooperative; back to producer
25 end
26 return  $y$ 
```

Table 1: Verifier cost spectrum. Times are typical per-call on a single CPU core (verifier) or on an H100 (LM-as-verifier, batch=1, $K=1$ token). Per-token LM cost is $\sim 10\text{--}30$ ms for a 7B-class model. The async / sync prediction is whether per-call verifier cost exceeds per-token LM cost.

Verifier	Used by	Typical per-call cost	Async pays?
Python <code>compile()</code>	ROCODE [12]	~ 0.1 ms	no
<code>g++ -fsyntax-only</code>	ROCODE [12]	~ 3 ms	marginal
SemGuard 1.3B classifier	SemGuard [22]	~ 10 ms	marginal
LSP completions (Java JDT)	MGD [1]	$\sim 20\text{--}80$ ms	yes
<code>cargo check</code> (incremental)	SoundCode (this work)	$\sim 35\text{--}500$ ms	yes
<code>javac</code> (warm JVM)	—	~ 180 ms	yes
LSP completions (rust-analyzer)	SoundCode (unevaluated)	$\sim 50\text{--}300$ ms	yes
Whole-crate borrow / trait analysis	— (unevaluated)	~ 500 ms—several s	strongly yes

`compile()`; warm single-file `g++` parses); SoundCode’s contribution is to show what happens when this inequality flips. In our decoding loop, per-invocation `g++` cost amortizes to ~ 0.55 s (§4.4), which already flips the inequality on C++; on Rust, `cargo check` amortizes to ~ 0.20 s — $\sim 7\text{--}20\times$ a per-token decoding step; the trade-off is no longer marginal. Section 4 measures these per-language costs empirically and confirms that async delivers wall-time wins at the ~ 150 ms-or-more per-call verifier cost regime (`g++ -fsyntax-only` amortized at ~ 0.55 s and `cargo check` amortized at ~ 0.20 s in our setup — both above the per-token decode budget of $\sim 10\text{--}30$ ms).

3.4 When to roll back

The decision to roll back depends on two predicates:

1. the verifier diagnostic is classified as BLOCKING (we define the classifier below);

2. the rollback target r returned by the policy is *strictly less than the current buffer length* $|y|$.

The second condition is essential in the async setting and is absent from synchronous baselines. In a synchronous loop, the verifier only sees the buffer at boundary b and returns before any further tokens are emitted, so any rollback target $r \leq b$ is by construction earlier than the current position. In the async setting, the verifier may report on a snapshot taken at boundary b_1 while the LM has already emitted tokens past a later boundary b_2 ; if a subsequent rollback to $r > b_1$ has already been applied (because, say, the user manually aborted, or because the LM emitted EOS), then the verifier’s verdict refers to already-discarded state and must be ignored.

Diagnostic classification. Following the synchronous-rollback literature [12, 22], we partition diagnostics into three classes:

- **INCOMPLETE:** errors caused by the buffer being a syntactic prefix of a valid program — missing closing brace, unfinished expression, parser “expected token X.” These are *not* rollback triggers; the LM is allowed to continue and the verifier will be re-invoked at the next boundary. Rust-side: syntax errors with span at end-of-buffer, plus E0282 “type annotations needed” if the buffer ends in a let-binding whose RHS has not yet been used. C++-side: parser-recovery errors at end-of-buffer.
- **NONBLOCKING:** warnings (`unused_variable`, `dead_code`) and style suggestions. These advance the checkpoint ratchet but do not trigger rollback.
- **BLOCKING:** everything else — name resolution failures, type mismatches reaching the expression top, borrow rule violations, trait-resolution failures. These trigger rollback.

We additionally argue that this classification, not the rollback algorithm itself, is the most fragile design choice in the system. A too-permissive classifier (everything is INCOMPLETE or NONBLOCKING) collapses SoundCode to vanilla decoding; a too-strict classifier (everything is BLOCKING) collapses it to whole-program retry. We treat the classifier as a hyperparameter, but the planned sweep is deferred; all experiments in this paper use a single fixed compiler-derived classifier (§4.7).

3.5 Where to roll back: rollback policies

ROCODE [12] introduced two rollback targets: r_e , the error-line offset reported by the compiler; and r_h , the line containing the highest-entropy token in the prefix [12, Eqs. 5–7]. They are tried in order: r_e on first error, r_h on recurrence. SemGuard [22] rolls back to the start of the offending line. IterGen [23] parameterizes the target by grammar non-terminal name.

We unify all four under a *rollback policy* π that maps (buffer, diagnostic, checkpoint set) to a token offset. Four instantiations are defined below; all four are now evaluated empirically (NAIVELAST and RECBACKOFF in Phase 1; the fully-wired RCODEENTROPY and NAIVELAST+PENALTY in the post-deadline follow-up, §4.6). The punchline of the ablations (§4.5, §4.6) is that naive last-checkpoint is the right default on this benchmark: both the recurrence-gated back-off and the full entropy rule tie it on C++ and trail it on Rust.

NAIVELAST (default, and the empirical winner). Return $\max C$, the most recent boundary checkpoint. As implemented, checkpoints are recorded *eagerly* when a boundary fires, not when that boundary’s verifier verdict lands: the first rollback for an error therefore lands at the boundary at which the offending statement was flagged, and each successive failed retry pops one checkpoint, backing off one boundary at a time. (A stricter variant — advancing checkpoints only on NONBLOCKING verdicts, so rollback targets are always verifier-cleared — is a natural alternative we did not evaluate; both schedulings share the eager driver, so the sync-async comparison is unaffected.) This is the simplest policy and the one we recommend on HumanEval-class benchmarks. The intuition is that with a strong code LM, most problems never trigger a rollback at all (132/156 Rust and 155/161 C++ problems under AsyncCargo-Naive; Section 4.5), and on the firing minority NAIVELAST settles within one or two retries, while the more aggressive targets re-encounter errors and thrash — the simpler target performs strictly no worse.

RECBACKOFF (evaluated). NAIVELAST augmented with RCODE’s recurrence *test*: when the verifier reports the same error code at the same source line as the previous rollback, back off one *additional* checkpoint (roll back to the second-most-recent boundary rather than the most recent).

This is the policy evaluated against NAIVELAST in Section 4.5; it hurt or tied NAIVELAST on this benchmark.

ROCODEENTROPY (evaluated in follow-up). ROCODE’s full two-rule formulation. First call: return the diagnostic’s error-line offset r_e . Second call with the same error at the same line: return r_h , the line containing $\arg \max_t H_t$, where H_t is the Shannon entropy of $\mathcal{M}$ ’s distribution at position t [12, Eq. 5]. In Phase 1 only the recurrence detector was wired (RECBACKOFF above); the post-deadline follow-up feeds real per-token entropies from the sampler’s top-20 logprobs and evaluates the complete rule (§4.6) — it ties RECBACKOFF on both languages.

NAIVELAST+PENALTY. NAIVELAST for target selection, plus ROCODE’s λ^{t-r} token penalty [12, Eq. 8–9] applied to all tokens after r in the retry distribution. The intent is to discourage immediate re-emission of the same erroneous prefix. Evaluated in the post-deadline follow-up (§4.6): a null at the rollback rates this benchmark induces.

Future work. Three natural extensions are out of scope for the present paper but supported by the same algorithm. (a) *Max-prob DFS*: maintain a set of candidate rollback targets, expand the one with the highest LM-conditional-probability suffix, and backtrack on recurrence. (b) *Beam-search rollback*: keep K checkpoints live in parallel, accept the first that the verifier blesses end-to-end, related to LATS [26]. (c) The *oscillation-detection* heuristic from week 5 of this project, in which N consecutive rollbacks to within an offset window of each other trigger escalation to a coarser target (halve the buffer, or restart from the prompt). We saw a concrete case in our experiments where the model emitted 66 syntactically-valid repetition statements before timing out at the 60 s budget; an oscillation detector would have caught this in tens of milliseconds (see §5).

3.6 Implementation

The reference implementation [25] is $\sim 8,500$ lines of Python and JavaScript. The producer is vLLM 0.20.x’s AsyncLLMEngine, used directly via its `generate()` async iterator. We register a vLLM LogitsProcessor for ROCODE-style penalty arms; for plain NAIVELAST, no logits processor is needed.

Verifiers are pluggable. The Rust path uses `cargo check` shelled out via `subprocess` on a per-problem workspace and parsed via the JSON message format; an LSP path using `multilspy` [1, 2] bound to `rust-analyzer 0.3.x` is exercised by the follow-up’s LSP arms (§4.6). For C++ we use `g++ -fsyntax-only` via `subprocess`. Each language has a tiny `BoundaryDetector` class (≤ 50 LoC) that returns boolean “is this position a depth-0 ;/}?” given the prefix tokens. Async coordination uses `asyncio` with one task per verifier call; on rollback all queued tasks are cancelled and their subprocesses sent `SIGTERM`.

The web demo [25] renders the producer buffer and the consumer’s verdict queue side-by-side with a WebSocket stream, animating rollback events to make the async control flow visible.

4 Results

4.1 Setup

Base model. Qwen2.5-Coder-7B-Instruct [10], served by vLLM 0.20.x with `bfloat16` weights and a 4096-token context. Greedy decoding at $T=0$, `max_tokens=512` per generate call (a problem that rolls back restarts generation and may emit more than 512 tokens in total), with a per-problem wall-clock budget of 60 s. The 60 s cap is on the entire problem, including verifier wall time; over-budget problems are counted as failures.

Benchmarks. MultiPL-E HumanEval-rs and HumanEval-cpp [4, 6]. The source benchmark has 164 problems; MultiPL-E ships 156 Rust and 161 C++ translations (problems that do not translate cleanly are dropped by the benchmark), and we use every shipped problem. For each problem, the prompt is the function signature plus a docstring describing the task; the completion is the function body. We evaluate `pass@1` by running the unit tests packaged with each problem against the generated body.

Hardware. A single workstation with one NVIDIA RTX PRO 6000 Blackwell (96 GB VRAM) for the LM and a 16-core CPU for the verifier. `cargo check` runs on a single core in incremental mode with the workspace warm. We pre-warm cargo on each language so the first verifier call of a session is not slower than the steady state.

Arms. The full design space factors into nine arms across the scheduling choices, the rollback policies, and the verifier: **Vanilla** (no verifier), **SyncCargo** (synchronous, ROCODE-style) and **AsyncCargo** (SoundCode) each crossed with the NAIVELAST / RECBACKOFF / NAIVELAST+PENALTY rollback policies (all with `cargo check / g++ -fsyntax-only` as verifier), plus **AsyncLSP** (rust-analyzer via `multilspy` as verifier, NAIVELAST policy) and **ROCODE-upstream** (the published ROCODE implementation).

Of these nine, the Phase-1 grid reports **four**, each run on both languages: Vanilla, SyncCargo + NAIVELAST, AsyncCargo + NAIVELAST, and AsyncCargo + RECBACKOFF. Every Phase-1 measurement (Table 2 and §§4.2–4.5) is therefore *compiler-supervised*, and the Phase-1 rollback-policy ablation compares NAIVELAST against RECBACKOFF only. A post-deadline follow-up (§4.6) adds the arms the original scoping cut: the fully-wired ROCODEENTROPY policy (async, both languages), AsyncLSP plus a SyncLSP companion (rust-analyzer as verifier), all four NAIVELAST+PENALTY arms, a Java third language, and a matched-budget resampling baseline outside the original nine. Still unevaluated at this revision: SyncCargo + RECBACKOFF and ROCODE-upstream (skipped for the separate reason documented in §4.8).

Metrics. `pass@1`, mean wall-clock time per problem (seconds), mean rollback count per problem, and compile rate (whether the final buffer `cargo check-passes / g++ -fsyntax-only-passes`). We additionally report *verifier idle time* (seconds spent awaiting a verdict that already blocks decoding) for the sync arms and the corresponding *producer wait fraction* for the async arms, to make the saved wall-time directly attributable to async overlap.

Runner revision (v2). During the follow-up we found a bug in the boundary detector’s firing logic: because trailing whitespace hides the statement terminator, a purely-whitespace token chunk re-fired the boundary that had already been checked, re-invoking the verifier (and, in sync mode, aborting the stream) on the *same* boundary. The tax fell asymmetrically on the sync arms — async’s single-in-flight-check rule absorbs re-fires — inflating sync wall-times by roughly 24% on Rust and 46% on C++ and overstating the async advantage. All numbers in this revision come from the corrected runner (v2). Removing the spurious aborts also shifts a modest number of per-problem outcomes through altered rollback interactions (most visibly sync-C++, where 14 test outcomes change and `pass@1` rises from 71.4% to 75.2%). For the record, the uncorrected runner measured $1.59\times$ (Rust) and $2.95\times$ (C++); the corrected headline below is $1.47\times$ and $2.16\times$. Both raw datasets are committed.

4.2 Main results

Table 2 summarizes the per-arm aggregates; Figure 6 visualizes the wall-time column. We isolate the sync-versus-async axis at fixed model, fixed verifier, and fixed rollback algorithm by comparing rows SyncCargo Naive vs AsyncCargo Naive.

Statistical note. All results are from a single greedy generation per problem per arm ($T=0$, one seed), so we report paired-difference significance rather than across-run variance. By McNemar’s exact test on paired problems, the headline sync-vs-async comparison at matched NAIVELAST policy shows no significant `pass@1` difference on either language (Rust: 3 vs 10 discordant, $p=0.092$, direction favoring sync; C++: 8 vs 5, $p=0.58$), and no verified arm differs significantly from Vanilla on `pass@1` (closest: SyncCargo-Rust at +6 problems, $p=0.15$, and RECBACKOFF-Rust at -6 , $p=0.070$). The only significant pairwise effects in the grid disfavor RECBACKOFF on Rust compile rate (vs Vanilla 1 vs 10, $p=0.012$; vs naive-async 1 vs 9, $p=0.022$). None of these effects favors verification over plain decoding. Accordingly, this paper claims a *latency* improvement only: the wall-time gaps in Table 2 are large, consistent across mean, median, and p90 (p90: 1.97 s sync vs 1.14 s async on Rust; 3.72 s vs 1.72 s on C++), and hold on 78–99% of individual paired problems; the pipeline’s run-to-run stability is characterized in §4.10.

Table 2: Per-arm Phase 1 summary. **Sched.** is the verifier-loop scheduling (**plain**: no verifier; **sync**: ROCODE-style block-and-wait; **async**: SoundCode producer-consumer); **Rollback** is the policy when a blocking diagnostic fires. **pass@1** is compile $\wedge$ tests-pass on the official MultiPL-E test cases; **comp.** is compile rate; **wall** is per-problem wall-clock; **rb** is rollback count; **toks** is generated tokens. Bold marks the fastest verified (sync/async) arm’s wall-clock per language; no verified arm significantly improves pass@1 or compile rate over **Vanilla** (see *Statistical note*). The ROCODE-UPSTREAM-CPP arm was skipped — see footnote.

Arm	Lang	Sched.	Rollback	n	pass@1	comp.	mean wall	median	mean rb	mean toks
Vanilla	Rust	plain	–	156	69.9%	86.5%	0.704	0.619	–	70.3
SyncCargo Naive	Rust	sync	naive	156	73.7%	87.8%	1.051	0.942	0.17	69.7
AsyncCargo Naive	Rust	async	naive	156	69.2%	85.9%	0.714	0.630	0.17	69.7
AsyncCargo RecBackoff	Rust	async	recb	156	66.0%	80.8%	0.892	0.654	0.74	70.1
Vanilla	C++	plain	–	161	77.0%	96.3%	0.865	0.770	–	86.5
SyncCargo Naive	C++	sync	naive	161	75.2%	93.2%	2.495	2.411	0.07	87.2
AsyncCargo Naive	C++	async	naive	161	77.0%	96.9%	1.156	1.091	0.06	86.2
AsyncCargo RecBackoff	C++	async	recb	161	76.4%	96.3%	1.271	1.089	0.26	86.3

The ROCODE-UPSTREAM-CPP arm is skipped: the released ROCODE implementation only ships a Python program analyzer, and porting its `PA_tools.py` to C++ is out of scope for this deadline (see §4.8). The SyncCargo Naive arms above are the algorithmic equivalent on the same model and the same verifier.

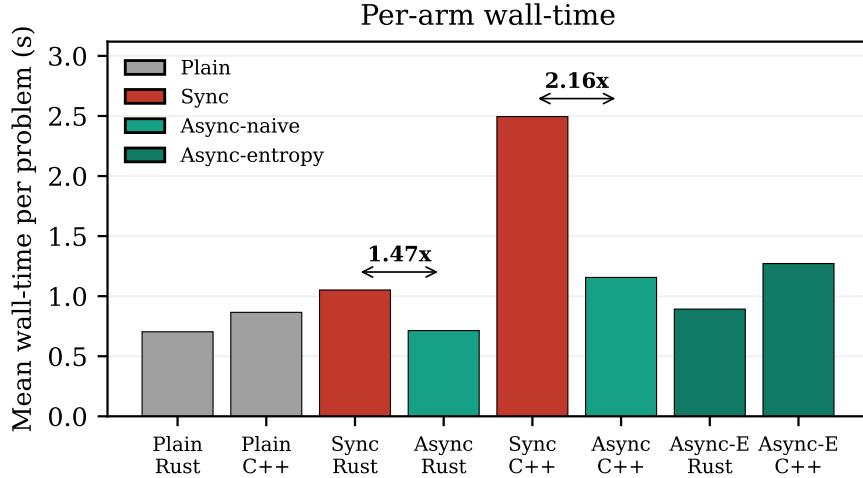


Figure 6: Mean wall-clock time per problem, per arm. **sync** (red) and **async** (teal) bars share the same model, the same verifier, and the same rollback algorithm; they differ only in scheduling. The annotated speedups are the ratio of means $\bar{t}_{\text{sync}}/\bar{t}_{\text{async}}$ for the naive-rollback pair in each language: $1.47\times$ on Rust, $2.16\times$ on C++. Plain (gray) is the verifier-free baseline.

4.3 Headline finding

C++. Switching the same rollback algorithm from sync to async cuts mean per-problem wall-time from 2.50 s to 1.16 s ($2.16\times$ speedup at the means; $2.20\times$ at the per-problem median). On 98.8% of paired problems (159 of 161), async is strictly faster than sync on the same problem (Figure 7). pass@1 is 77.0% async vs 75.2% sync, a difference of a handful of problems that is not statistically significant (McNemar exact test on the paired problems, 8 vs 5 discordant, $p=0.58$), and we do not claim it as an accuracy effect.

Rust. The wall-time win on Rust is $1.47\times$ (1.05 s sync to 0.71 s async; $1.31\times$ at the per-problem median), and the async arm’s overhead over vanilla decoding is just 1.4% (0.714 s vs 0.704 s) — on Rust, verification is essentially free under async scheduling. Async beats sync on 78.2% of paired problems (122/156). pass@1 is 73.7% sync vs 69.2% async; as on C++, the difference is not statistically significant (3 vs 10 discordant, McNemar exact $p=0.092$). The smaller wall-time

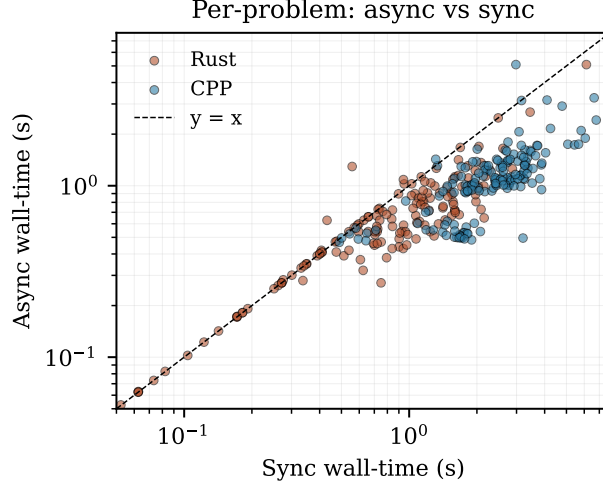


Figure 7: Per-problem sync wall-time (x) vs async wall-time (y), one dot per problem. Dots below the diagonal are problems on which async is faster than sync. The C++ cloud sits visibly below the diagonal at every cost level; the Rust cloud crowds the diagonal at the cheap end (where the verifier is fast and sync is already adequate) but still skews under it.

gain is exactly the verifier-cost-spectrum prediction of §3.3: incremental `cargo check` on the warm workspace amortizes to ~ 0.20 s per call in our setup (§4.4) — roughly a third of C++’s per-check cost — so the sync arm’s per-boundary stall is shorter, leaving less wall-time for the async overlap to recover.

4.4 Rust vs. C++: the verifier-cost spectrum

The asymmetry $2.16\times$ (C++) vs $1.47\times$ (Rust) is predicted by the cost-spectrum analysis of §3.3: the relative async win scales with the ratio c_v/c_d of verifier cost c_v to per-token decode cost c_d .

Concretely, the sync arm’s wall-time decomposes as $\bar{t}_{\text{sync}} \approx \bar{t}_{\text{plain}} + c_v \cdot K$, where K is the number of boundary checks per problem. The async arm pushes the $c_v K$ term off the critical path entirely, so its wall-time asymptotically returns to $\bar{t}_{\text{plain}}$. Empirically, this matches the data on C++: plain C++ is 0.87 s and async-C++ is 1.16 s (a 34% overhead from rollback + the end-of-stream drain), while sync-C++ is 2.50 s, dominated by $K \approx 3.0$ g++ syntax-only invocations at ~ 0.55 s amortized cost each (the cold-start time of the C++ compiler is large relative to anything `cargo check` pays). On Rust, plain is 0.70 s and async-Rust is 0.71 s (a 1.4% overhead); the sync overhead is ~ 0.35 s ($K \approx 1.8$ checks at ~ 0.20 s amortized incremental `cargo check`). Figure 9 visualizes this decomposition.

4.5 Ablation: rollback policy

Within the AsyncCargo arms, the recurrence-gated back-off (RECBACKOFF, §3.5) does *not* improve over the simpler NAIVELAST policy on this benchmark: on Rust, RECBACKOFF is 66.0% pass@1 vs naive 69.2% (and compiles significantly less often: 126 vs 134, $p=0.022$); on C++, it is 76.4% vs naive’s 77.0%. Rollback is rare overall — 132 of 156 Rust problems and 155 of 161 C++ problems never roll back under AsyncCargo-Naive (0.17 and 0.06 mean rollbacks/problem; Figure 8) — and the two policies diverge sharply on the firing minority: under NAIVELAST a firing problem rolls back once or twice (median 1, max 2) and settles, while under RECBACKOFF the extra back-off re-encounters errors and typically thrashes to the per-problem rollback budget (median 7 among firing problems, the budget maximum). The clean read is that the simpler policy suffices on this benchmark: most problems never exercise the policy at all, and the more aggressive target actively harms the minority that do. The same conclusion holds for ROCODE’s full entropy-based target selection, evaluated in §4.6.

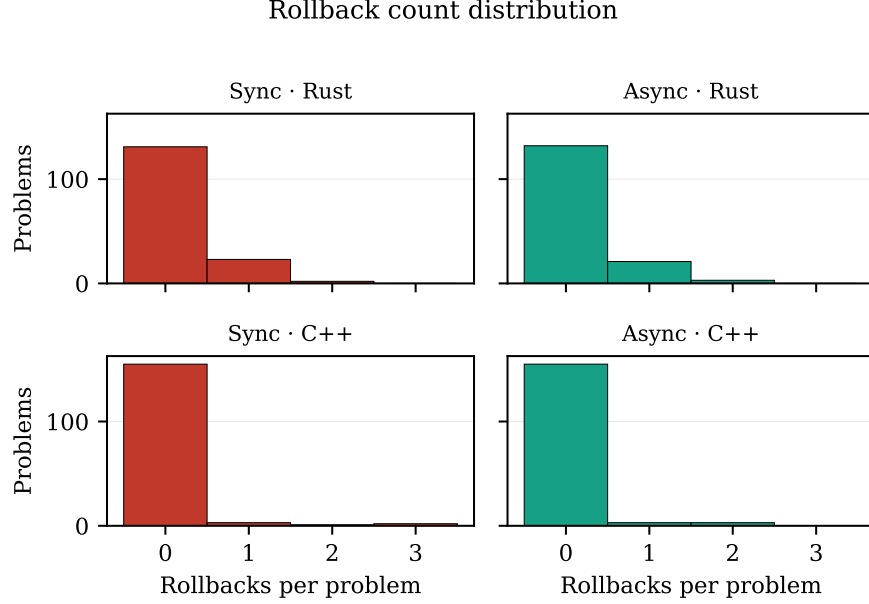


Figure 8: Distribution of rollback counts per problem, faceted by language $\times$ scheduling. The mass is concentrated at 0 rollbacks (no blocking diagnostic ever fires); firing problems settle within one or two retries under NAIVELAST in both schedulings.

4.6 Extended arms (post-deadline follow-up)

The arms below were run after the original submission, on the same model, hardware, benchmark revision, and decoding configuration as Phase 1; raw records are committed alongside the Phase-1 data.

ROCODEEntropy, fully wired. We wired RCODE’s complete target-selection rule into the sampler — per-token Shannon entropies from the top-20 logprobs, error-line offset r_e on the first rollback for an error, max-entropy line r_h on recurrence — and re-ran the async arms. The verdict from the RECBACKOFF proxy stands. On C++, the wired rule ties RECBACKOFF (123/161 pass@1) one problem below NAIVELAST (124). On Rust it scores 66.0% (103/156) — identical to RECBACKOFF, non-significantly below NAIVELAST on pass@1 (1 vs 6, $p=0.13$), significantly below it on compile rate (1 vs 9, $p=0.022$), and significantly below Vanilla on pass@1 (0 vs 6, $p=0.031$). Entropy-targeted rollback does not rescue the recurrence-heavy minority on this benchmark; it only relocates the rollback target inside problems that fail under every policy. (The wired arm’s wall-times, 1.23 s Rust / 1.39 s C++, are not comparable to Table 2: computing top-20 logprobs slows decoding for every token.)

LSP verification. With the repaired rust-analyzer driver (below), we ran the AsyncLSP arm — and a SyncLSP companion — on Rust. Three results. *First, the LSP tier is not vestigial once the driver is correct:* blocking diagnostics fire and roll back (6 rollbacks across 6 problems async; 15 across 13 sync), in contrast to earlier project experiments in which the LSP tier never blocked. Much of that earlier null result traces to a driver bug rather than to rust-analyzer itself: the shipped multilspy-based checker never delivered content updates the server acted on (rust-analyzer computes rustc-class diagnostics via its embedded cargo check only on didSave, which the driver never sent), so every check after the first returned the first check’s diagnostics. *Second, accuracy tracks the compiler arms exactly:* async-LSP passes 108/156 (async-cargo’s count; vs Vanilla $p=1.0$), sync-LSP 115/156 (sync-cargo’s count; vs Vanilla $p=0.15$). *Third, the sync-vs-async contrast reproduces at LSP-grade verifier cost:* 3.52 s sync vs 2.46 s async at the means (1.43 $\times$; async faster on 128/156 paired problems, median per-problem ratio 1.33 $\times$). Two honest caveats. LSP verification is strictly dominated by cargo check here — same pass@1 at 3.4 $\times$ async-cargo’s wall-time (2.46 s vs 0.71 s) — and the 1.43 $\times$ ratio sits *below* even Rust-cargo’s 1.47 $\times$ despite a per-check cost roughly ten times

higher, which refines the cost-spectrum model of §3.3: once a single check ($\sim 1\text{--}2$ s) approaches the total decode time, the final end-of-stream verdict — which async must await for correctness — cannot be hidden, and the async win saturates. The prediction “async pays when per-check cost exceeds per-token cost” holds; the win is largest when per-check cost is also *small relative to total decode time*.

Model-size scaling. The cost-spectrum framing (§3.3) makes a falsifiable prediction: the async win grows with the ratio of per-check verifier cost to decode cost, so a smaller, faster-decoding model should show a *larger* win at the same verifier. We re-ran the four core arms with Qwen2.5-Coder-1.5B-Instruct (vanilla decode: 0.20 s/problem Rust, 0.23 s C++ — $3.6\text{--}3.8\times$ faster than the 7B). The prediction holds on both languages: the sync/async ratio rises from $1.47\times$ to $1.69\times$ on Rust (async faster on 126/156 paired problems) and from $2.16\times$ to $3.17\times$ on C++ (159/161), with pass@1 again statistically indistinguishable between schedulings (McNemar $p = 0.27$ Rust, $p = 1.0$ C++). (The 1.5B grid ran on the corrected runner; its ratios are directly comparable to the v2 headline.) The spectrum’s measured points are ordered exactly as per-check-cost relative to decode time predicts.

A second benchmark: MBPP. HumanEval’s n leaves ± 7 pp confidence intervals on pass@1; MultiPL-E’s MBPP translations give $2.3\times$ the sample ($n = 354$ Rust, $n = 397$ C++). Both headline findings survive the added power. The latency result: sync/async ratio-of-means $1.54\times$ on Rust (async faster on 273/354 paired problems) and $2.18\times$ on C++ (394/397), at means $0.91\text{ s} \rightarrow 0.59\text{ s}$ and $2.25\text{ s} \rightarrow 1.03\text{ s}$. The accuracy null sharpens rather than weakens: sync-vs-async pass@1 is non-significant on both languages (15 vs 10 discordant, $p = 0.42$ Rust; 11 vs 5, $p = 0.21$ C++), and async-vs-Vanilla on C++ has *zero* discordant problems out of 397 — the verified arm passes exactly the problems vanilla decoding passes. The bar any in-loop verification system must clear [17] is post-hoc filtering under the same budget. Our baseline samples complete candidates at $T = 0.8$ (seeded per sample) and submits the first that *compiles* — the compiler as a post-hoc filter rather than an in-loop supervisor; unit tests are never consulted during selection. On Rust it is the strongest arm in this paper: 120/156 pass@1 (76.9%) at 1.12 s mean wall and 1.39 samples per problem — significantly above async in-loop verification on both pass@1 (20 vs 8 discordant, $p = 0.036$) and compile rate (155/156; 21 vs 0, $p < 0.001$), and above Vanilla on pass@1 at the edge of significance ($p = 0.052$). On C++ it trails on pass@1 within noise (120/161 vs async’s 124, $p = 0.52$; vs Vanilla $p = 0.54$) with a compile rate of 160/161 (vs async 4 vs 0 discordant, $p = 0.13$) at higher wall (1.87 s vs 1.16 s). Two qualifications: the baseline decodes at $T = 0.8$ while the verified arms are greedy, so this is a system-level comparison, not a one-variable ablation; and it emits $1.4\text{--}1.5\times$ the tokens (whole candidates are discarded, not suffixes). The honest synthesis: *for batch pass@1 on this benchmark, resample-and-filter is the stronger default on Rust and competitive on C++, and the case for in-loop verification rests on what resampling cannot provide — a monotonically growing verified prefix (streaming/interactive use), bounded per-problem tail latency, and the larger salvage value of long generations, none of which this benchmark measures.*

NaiveLast+Penalty. We completed the last policy in the design space by implementing ROCODE’s λ^{t-r} token penalty as an engine-registered per-request logits processor (vLLM V1’s subprocess architecture rules out the shared-state processor the original implementation attempted; the penalty trie is serialized into each retry request instead). The result is a null: against the corrected NAIVELAST baselines, the penalty changes pass@1 by at most one problem in any arm (all McNemar $p = 1.0$; e.g. async-Rust 109 vs 108, async-C++ 124 vs 124) at indistinguishable wall-times. With rollback firing on $\sim 4\text{--}15\%$ of problems and settling within one or two retries, the mask simply has almost nothing to steer. We note this expressly because an earlier uncontrolled comparison (penalty on the corrected runner vs naive on the uncorrected one) made the penalty look transformative; against the matched baseline the effect disappears.

Java. A third language probes the javac-grade region of the cost spectrum (MultiPL-E humaneval-java, $n = 158$; javac in-loop amortizing to ~ 0.2 s per invocation). Async verification behaves as on Rust and C++: 107/158 pass vs vanilla’s 108, at 0.83 s vs 0.67 s mean wall (+24%). The sync arm, however, degrades outright (86/158 pass, 107/158 compile): its per-boundary abort-and-restart cycle drifts the model’s indentation, nested closing braces land at the four-space indent level, and MultiPL-E’s indentation-sensitive stop token (`\n` `}`) then truncates the function mid-body — an artifact of restart-based scheduling interacting with stop-token conventions, not of verification. We therefore do not quote a sync/async ratio for Java; the honest Java result is that *async verification*

is accuracy-neutral at modest overhead where sync scheduling is not even well-defined under the benchmark’s stop-token convention.

4.7 Ablation: diagnostic classifier

The diagnostic-classifier hyperparameter (§3.4) was originally going to be swept against a learned-classifier baseline (SemGuard [22]); however, a faithful SemGuard comparison requires retraining the 1.3B classifier on a curated Rust diff corpus (the released SemGuard model is Python/Java only), which is out of scope for this paper. We therefore use a fixed compiler-derived classifier (BLOCKING = any error[E...] diagnostic) across all sync and async arms, and defer the classifier-threshold sweep to the SemGuard-comparison follow-up described in §5.

4.8 RCODE-upstream baseline

The RCODE-upstream arm, a head-to-head against the published RCODE [12] implementation on HumanEval-cpp, was not runnable in time. The released RCODE codebase ships a Python-only program analyzer (PA_tools.py, which calls Python’s `exec()` and catches `SyntaxError` / `AssertionError`); it has no C++ analyzer and no hook to substitute one. Porting RCODE’s PA to C++ (a g++ syntax-error parser plus a dynamic test driver) is a multi-day engineering effort that we judged not worth its share of the wall-clock budget, since the algorithmic comparison we care about is already isolated by the SyncCargo Naive arms. Those arms run RCODE’s algorithm *exactly* — block at every depth-zero structural boundary, run the program analyzer to completion, classify the verdict, roll back to the most recent boundary checkpoint on a blocking error — on the same Qwen2.5-Coder-7B model and the same g++ -fsyntax-only/cargo check verifier the async arms use. The only differences from upstream RCODE are the language adapter (C++/Rust rather than Python) and the inference engine (vLLM rather than HuggingFace transformers). We document this scoping decision in results/paper_phase1/known_issues.md.

4.9 Producer-wait fraction

Figure 9 decomposes each arm’s mean wall-time into a *decode* term (estimated as the corresponding vanilla-arm mean) and a *verifier-overhead* term (the residual, including all serialized verifier wait and rollback wall-time). On C++, the verifier-overhead share is 65% of sync wall-time and drops to 25% for async; on Rust, 33% shrinks to 1.4%. The async verifier-overhead is non-zero because (i) the producer must still await the final verifier verdict before emitting the EOS token (the correctness invariant of §3.1), and (ii) any actual blocking rollback incurs the discarded suffix’s decode time.

4.10 Reproducibility

All artifacts needed to regenerate this paper’s numbers are in the repository: the experiment runner (scripts/paper_phase1.py), the analysis script (scripts/paper_phase2.py), the raw per-problem JSONL records and run log, and a locked Python environment (uv.lock: vLLM 0.20.2, torch 2.11.0, datasets 4.8.5). The model is the stock HuggingFace snapshot of Qwen2.5-Coder-7B-Instruct; the benchmark revision of nuprl/MultiPL-E is pinned in the runner. The full grid runs in ~40 minutes on the hardware of §4.1 (uv run python scripts/paper_phase1.py, then scripts/paper_phase2.py for aggregates and figures).

Verification and replication. The pipeline’s repeatability was established on the original (uncorrected) runner before the v2 revision: a 20-problem-per-language re-execution of all eight arms matched the stored records exactly on 160/160 per-problem compile/test/token outcomes with per-arm mean wall-times within 3%, and five independent full-grid executions agreed to $\pm 0.03\%$ (Rust) and $\pm 0.2\%$ (C++) on the headline ratio with per-problem outcomes identical across all five runs — including the async arms’, which are race-dependent in principle. Raw records of every run, v1 and v2, are committed. The analysis pipeline is deterministic: re-running it regenerates the aggregate CSV and report byte-identically and all figures content-identically.

Determinism caveats. Plain and sync arms are token-level deterministic up to vLLM prefix-caching numerics (greedy $T=0$, problems served sequentially). Async-arm outcomes depend in principle on verifier-vs-decoder wall-time races (whether a verdict lands before token k or $k+1$), so their rollback

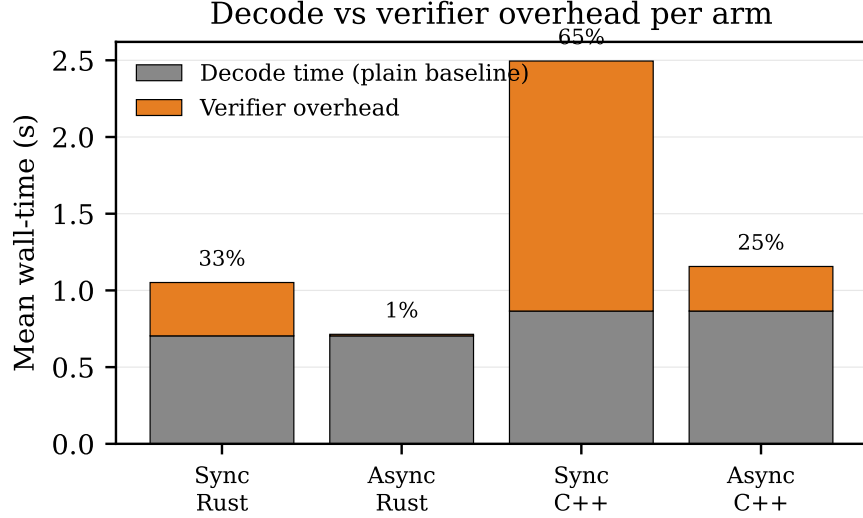


Figure 9: Per-arm wall-time decomposed into *decode* (the vanilla-arm mean, gray) and *verifier overhead* (the residual, orange; serialized verifier wait + rollback decode). Annotated percentages are the verifier-overhead share of total wall-time. Async pushes the orange term from 65% to 25% on C++ and from 33% to ~1% on Rust.

offsets and generated code reproduce approximately rather than by construction — though they reproduced exactly in the verification above. All wall-time magnitudes are specific to the hardware and toolchain; the runner logs `rustc`, `cargo`, and `g++` versions with each run (verification used `rustc` 1.92.0 and `g++` 13.3.0).

5 Conclusion

We presented SoundCode, the first asynchronous design for verifier-guided code generation. The system is a producer-consumer decoding loop in which a compiler verifier runs in a background task at each depth-zero structural boundary, the producer LM streams tokens without blocking, and a consumer rolls back the buffer to the most recent boundary checkpoint only on a returned blocking diagnostic. The same rollback algorithm underlies the synchronous ROCODE-style baseline [12]; the sync-versus-async axis is isolated at fixed verifier, fixed model, and fixed benchmark.

Where async pays. The win grows with verifier latency. Empirically, on Rust — where `cargo check` is $\sim 7\text{--}20\times$ a per-token decoding step in our setup — async overlap recovers essentially all of the verifier wall-time ($1.47\times$ speedup; async wins on 122 of 156 paired problems; 1.4% overhead over vanilla). On C++, where each `g++ -fsyntax-only` invocation amortizes to ~ 0.55 s of per-boundary cost, the win is larger still ($2.16\times$; async wins on 159 of 161 paired problems) — as the cost-spectrum analysis in Section 3.3 predicts, and as replicates on MBPP ($1.54\times/2.18\times$) and at 1.5B ($1.69\times/3.17\times$). `pass@1` is statistically indistinguishable between the schedulings on both languages (§4.2); the result is a latency result. On Java, async verification is likewise accuracy-neutral at +24% wall, while restart-based sync scheduling collides with the benchmark’s stop-token convention (§4.6); whole-crate borrow / trait analysis (up to seconds per check) remains unevaluated, though the same architecture supports it without modification.

Limitations. The matched-budget caveat from Olausson et al. [17] applies: any speedup is only meaningful if the verifier itself is a strictly-stronger oracle than the generator on the disputed fragment. We argue that a compiler *is* such an oracle on the decidable fragment of syntax, type, and lifetime correctness — unlike a learned classifier [22], the compiler has no false-positive rate on errors that fall inside its formal spec. But this argument is restricted to that decidable fragment: the compiler tells us nothing about functional correctness, and any test-pass gain SoundCode delivers over vanilla decoding has to come from compilable-but-wrong code being a small fraction of all generated

code. We do not address this gap; SemGuard [22] is the natural complement that targets exactly the semantic-but-compilable failure mode.

The current evaluation is also single-model and single-seed, and we accordingly make no accuracy claims: none of the pass@1 differences between arms is statistically significant (§4.2), and the contribution of this paper is confined to wall-clock latency. The post-deadline follow-up (§4.6) closes the original scoping gaps — the AsyncLSP arm (the sync-async contrast reproduces at $1.43\times$, with the win saturating once a single check approaches total decode time), the fully-wired ROCODEENTROPY policy, the PENALTY policy (a null at these rollback rates), a Java third language, grid replications, and a matched-budget resampling baseline that *beats* in-loop verification on Rust batch pass@1 — the last of which sharpens this paper’s scope: on batch evaluation, resample-and-filter is the stronger default on Rust, and in-loop verification’s case rests on streaming latency, bounded tails, and long-generation salvage that this benchmark does not measure. A faithful SemGuard retraining on Rust diagnostics remains the obvious next experiment.

Future work. Five extensions are immediate. (1) *Oscillation-detection rollback.* On HumanEval-140 (fix_spaces) under our SyncCargo arm, the model hit the 60 s problem budget after emitting 66 syntactically-valid repetition statements that all re-introduced the same compile error in the same place; each one was individually cleared by the boundary detector and re-rejected by the verifier. A policy that detects “ N consecutive rollbacks landing within an offset window of each other” and escalates to a coarser target (halve the buffer; restart from the prompt with a penalty mask) would have caught this pathology in tens of milliseconds. The case is concrete evidence that the next paper-sized contribution on this architecture is a *penalty / oscillation* mechanism layered on top of the rollback policy. (2) *Faithful SemGuard comparison.* SemGuard is the strongest learned-verifier baseline; adapting it from Python/Java to Rust requires retraining the 1.3B classifier on a curated Rust diff corpus, work currently in progress. (3) *Larger benchmarks.* MBPP-rs and LiveCodeBench [11] would extend the evaluation from single-function HumanEval to multi-function and contamination-free problems. (4) *Beam-search and DFS rollback policies.* Our unifying π abstraction admits multi-branch policies that we have not evaluated; the natural reference is LATS [26]. (5) *Composition with cheap front-end masks.* DOMINO [3] and Type-Constrained [16] are near-zero-overhead syntactic prevention layers; composing them with SoundCode’s expensive semantic verifier-with-rollback back-end should be Pareto-better than either alone.

Artifacts. A live web demo and the reference implementation are open-source. Demo: `soundcode-demo.psixyzt.com`. Source: `github.com/leobwang/soundcode` [25].

References

- [1] Lakshya A Agrawal, Aditya Kanade, Navin Goyal, Shuvendu Lahiri, and Sriram Rajamani. Monitor-guided decoding of code LMs with static analysis of repository context. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2306.10763>. arXiv:2306.10763.
- [2] Lakshya A Agrawal et al. multilspy: Cross-platform Python LSP-client library, 2023. Part of monitors4codegen; <https://github.com/microsoft/monitors4codegen>.
- [3] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. Guiding LLMs the right way: Fast, non-invasive constrained generation. In *International Conference on Machine Learning*, 2024. URL <https://arxiv.org/abs/2403.06988>. arXiv:2403.06988.
- [4] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, et al. MultiPL-E: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering*, 2023. URL <https://arxiv.org/abs/2208.08227>. arXiv:2208.08227.
- [5] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling, 2023. URL <https://arxiv.org/abs/2302.01318>. arXiv:2302.01318.

- [6] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code, 2021. URL <https://arxiv.org/abs/2107.03374>. arXiv:2107.03374.
- [7] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2304.05128>. arXiv:2304.05128.
- [8] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the sequential dependency of LLM inference using lookahead decoding. In *International Conference on Machine Learning*, 2024. URL <https://arxiv.org/abs/2402.02057>. arXiv:2402.02057.
- [9] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Empirical Methods in Natural Language Processing*, 2023. URL <https://arxiv.org/abs/2305.13971>. arXiv:2305.13971.
- [10] Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, et al. Qwen2.5-Coder technical report, 2024. URL <https://arxiv.org/abs/2409.12186>. arXiv:2409.12186.
- [11] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code, 2024. URL <https://arxiv.org/abs/2403.07974>. arXiv:2403.07974.
- [12] Xue Jiang, Yihong Zheng, Yuxiang Lyu, Changjian Niu, Bin Wang, Zibin Hu, and Zhi Jin. RCODE: Integrating backtracking mechanism and program analysis in large language models for code generation, 2024. URL <https://arxiv.org/abs/2411.07112>. arXiv:2411.07112.
- [13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *ACM Symposium on Operating Systems Principles*, 2023. URL <https://arxiv.org/abs/2309.06180>. arXiv:2309.06180.
- [14] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, 2023. URL <https://arxiv.org/abs/2211.17192>. arXiv:2211.17192.
- [15] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Prasad Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2303.17651>. arXiv:2303.17651.
- [16] Niels Mündler, Jingxuan He, Tobias Müller, Petar Tsankov, and Martin Vechev. Type-constrained code generation with language models. In *ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2025. URL <https://arxiv.org/abs/2504.09246>. arXiv:2504.09246.
- [17] Theo X Olausson, Jeevana Priya Inala, Chenglong Wang, Jianfeng Gao, and Armando Solar-Lezama. Is self-repair a silver bullet for code generation? In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2306.09896>. arXiv:2306.09896.
- [18] Gabriel Poesia, Alex Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. Synchromesh: Reliable code generation from pre-trained language models. In *International Conference on Learning Representations*, 2022. URL <https://arxiv.org/abs/2201.11227>. arXiv:2201.11227.
- [19] Tal Ridnik, Dedy Kredo, and Itamar Friedman. Code generation with AlphaCodium: From prompt engineering to flow engineering, 2024. URL <https://arxiv.org/abs/2401.08500>. arXiv:2401.08500.

- [20] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Empirical Methods in Natural Language Processing*, 2021. URL <https://arxiv.org/abs/2109.05093>. arXiv:2109.05093.
- [21] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2303.11366>. arXiv:2303.11366.
- [22] Zhihao Tian, Lei Yin, Bingyu Zhang, Hongwei Wang, Shaohua Zhang, Yulei Hou, Yibo Ye, and Yu Zhou. SemGuard: Real-time semantic evaluator for correcting LLM-generated code, 2025. URL <https://arxiv.org/abs/2509.24507>. arXiv:2509.24507.
- [23] Shubham Ugare, Rohan Gumaste, Tarun Suresh, Gagandeep Singh, and Sasa Misailovic. Iter-Gen: Iterative semantic-aware structured LLM generation with backtracking. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=ac93gRzxxV>.
- [24] Hanbin Wang, Zhenghao Liu, Shuo Wang, Ganqu Cui, Ning Ding, Zhiyuan Liu, and Ge Yu. INTERVENOR: Prompting the coding ability of LLMs with the interactive chain of repair. In *Findings of the Association for Computational Linguistics: ACL*, 2024. URL <https://arxiv.org/abs/2311.09868>. arXiv:2311.09868.
- [25] Leo Wang. SoundCode: A reference implementation of async LSP-supervised decoding. <https://github.com/leobwang/soundcode>, 2025.
- [26] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *International Conference on Machine Learning*, 2024. URL <https://arxiv.org/abs/2310.04406>. arXiv:2310.04406.